Derivation D3: Deterministic Simulation and Timestamp-Order Processing

Strict Total Ordering, Causal Monotonicity, and Invariant Preservation

Pranav (250001034) – Role P3 (Simulation Architect)
CS 253 / MA 253 Data Structures & Algorithms Lab

21 November 2026

1 Motivation: Eliminating Timestamp Ties

In discrete-event simulations, events frequently share identical timestamps. For example, at simulation tick $t = 50$, an acknowledgment packet arrival A and a retransmission timeout expiration B may coincide. If A executes first, timeout B is cancelled, preventing duplicate packet dispatch. If B executes first, an unneeded retransmission packet is scheduled, leading to state divergence.

To ensure bit-for-bit reproducibility and allow rigorous comparative evaluation across disparate priority queue implementations (Structure A, Structure B, and Sorted-Array Oracle), tie-breaking must follow a deterministic, mathematically rigorous total order.

2 Formal Tie-Breaking Rule

Every scheduled event e is assigned a 64-bit integer sequence number $s(e)$ from a monotonic global counter `nextSeq++`, initialized to 0 and incremented exclusively within `Simulator::schedule()`.

Definition (Lexicographical Total Order):
Event e precedes event f (written $e < f$) if and only if:

$$e < f \iff (t(e) < t(f)) \vee \big(t(e) = t(f) \wedge s(e) < s(f)\big) \tag{1}$$

Lemma 1 (Uniqueness of Sequence Numbers) *If e and f are distinct events ($e \neq f$), then $s(e) \neq s(f)$.*

Proof: Because `nextSeq` strictly increases by 1 per event scheduling call and does not wrap around within 2^{64} operations, each event receives a unique sequence number. ■

Lemma 2 (Strict Total Ordering) *The relation $<$ defines a strict total order over the universe of all events.*

Proof: We verify the four axioms of strict total order:

1. **Irreflexivity:** For any event e, $t(e) = t(e)$ and $s(e) = s(e)$; neither condition for $e < e$ holds.
2. **Asymmetry:** If $e < f$, then either $t(e) < t(f)$ (implying $t(f) > t(e)$) or $t(e) = t(f)$ and $s(e) < s(f)$ (implying $s(f) > s(e)$). In both cases, $f < e$ cannot hold.

3. **Transitivity:** If $e < f$ and $f < g$, then $t(e) \leq t(f) \leq t(g)$. If $t(e) < t(g)$, then $e < g$. If $t(e) = t(f) = t(g)$, then $s(e) < s(f) < s(g)$, so $e < g$.

4. **Trichotomy:** For distinct $e \neq f$, if $t(e) \neq t(f)$, timestamps order them. If $t(e) = t(f)$, by Lemma 1 $s(e) \neq s(f)$, so sequence numbers order them. Exactly one of $e < f$ or $f < e$ holds.

■

3 Simulation Determinism and Timestamp Monotonicity

Theorem 1 (Bit-for-Bit Trace Determinism) *Under identical initial configurations and random seeds, any correct implementation of* `EventSet` *produces an identical sequence of processed events.*

Proof: By mathematical induction on the simulation step k. At step k, the event set contains a unique minimal element under $<$ (by Lemma 2). Since deleteMin has no degrees of freedom, all structures extract the identical event e_k. The deterministic event handler processes e_k using our deterministic PRNG (`SplitMix64`), scheduling identical child events and issuing identical cancellations. Hence step $k + 1$ begins in an identical state across all implementations. ■

Theorem 2 (Timestamp Monotonicity / Causal Consistency) *Let $e_1, e_2, e_3, \ldots$ denote the sequence of processed events. Then $t(e_k) \leq t(e_{k+1})$ for all $k \geq 1$.*

Proof: All simulator actions require non-negative delay: $\Delta t \geq 1$ tick. If e_{k+1} was already present when e_k was popped, $t(e_k) \leq t(e_{k+1})$ since e_k was minimal. If e_{k+1} was scheduled while processing e_k, causality enforces $t(e_{k+1}) = t(e_k) + \Delta t \geq t(e_k)$. ■